

Ideas para el orquestador

Objeto

Este documento es un conjunto de ideas iniciales para el desarrollo del orquestador de máquinas virtuales

Antecedentes

Se parte de las librerías esxobjects y yamlinfr.

Ideas generales

Para darle una mayor modularidad y poder ser una solución más universal, se propone dividir el orquestador en 2 capas:

- Un núcleo de orquestación, el orquestador en sí. Esta capa organiza las máquinas virtuales en general, proporcionando una base universal para varios problemas.
- Una aplicación que utiliza el orquestador, especializada en la creación de conjuntos de VM para las docencia de asignaturas de la universidad. Se puede ver como una aplicación o como un módulo add-on, al orquestador.
- De esta forma, se podrán hacer más aplicaciones para otros problemas.

Módulo Orquestador.

Será un servicio linux, que se encarga de mantener la organización de VM. La idea es que los usuarios y aplicaciones superiores interactúen con el orquestador, como objeto de más alto nivel que interactuar directamente con los hipervisores. Ya que proporciona características adicionales, y un interfaz de más alto nivel.

Conceptos clave

- **Definición** de una VM: Se trata de la lista de todos los parámetros necesarios para desplegar una VM. En estos parámetros están tanto los parámetros que necesita el hypervisor, como otros que se decidan. Por ejemplo: imagen desde la que crear la VM, características hardware, ubicación del disco virtual, datos de red, etc.
- **VM**: la VM real que se ejecuta dentro de un hypervisor. Desde un punto de vista conceptual las VM “viven” en el hypervisor.
- **VM descriptor**. Es el concepto clave de este desarrollo. Es un interface de más alto nivel de la VM.
 - El orquestador, mantiene el listado de los descriptors. Los descriptors “viven” en el orquestador.
 - Tienen una referencia al hipervisor y a su VM de referencia.
 - El usuario sólo interactúa con el descriptor para realizar TODAS las funciones de administración de la VM correspondiente. La idea es que, desde el punto de vista del usuario, el descriptor representa la VM, de forma que sea “lo mismo”.
 - Podemos establecer una analogía entre estos descriptors y los ficheros de disco en un sistema operativo, en el sentido de que, un usuario ve un fichero de disco como un objeto con ciertos métodos. Pero no interactúa con su representación de bloques a nivel de partición de disco, ni se preocupa que cada fichero esté en una partición con un formato diferente. La idea es parecida, que el usuario vea todas las VM de forma similar. Pero el orquestador se preocupa de detalles de bajo nivel, como saber qué hipervisor se ejecuta, qué tipo de hipervisor es, y otros detalles.

Reflexiones acerca de la coherencia de datos.

Tenemos pues 3 entidades con información relacionada:

- La definición de una VM
- El descriptor de una VM
- La MV

Es deseable mantener una coherencia entre estos datos. Además, para el usuario del sistema, tiene que ser lo más sencillo posible. La solución propuesta pasa por que el usuario sólo vea un objeto. En este caso el descriptor. Así que, para simplificarlo, unificamos

la definición con el descriptor. Lo que hacemos es que el descriptor, también guarde la definición de la VM. Así, un descriptor es un objeto que hace varias cosas:

- guarda la definición de la VM
- tiene una referencia a la VM
- tiene los métodos necesarios para todo el ciclo de vida de la VM
- Un concepto importante es ver qué un descriptor estará en alguno de estos estados: (se podrá modelar como una máquina de estados):
 - **Provisioned**: El descriptor existe, pero la VM no ha sido instalada en el hipervisor. Es solo una definición o plantilla.
 - **Created**: La VM ha sido instanciada en el hipervisor con sus recursos, pero está apagada. Es su estado "frío".
 - **Starting**: La VM está en el proceso de encendido. Es un estado de transición.
 - **Running**: La VM está activa y en funcionamiento.
 - **Paused**: La VM está suspendida, pero su estado está guardado en memoria.
 - **Stopping**: La VM está en proceso de apagado. Es otro estado de transición.
 - **Stopped**: La VM ha sido apagada. Es idéntico al estado **Created** en la práctica, pero se llega a él después de haber estado en **Running**. A menudo, en la implementación, **Created** y **Stopped** pueden ser el mismo estado con nombres diferentes en el código.
 - **Failed**: Se puede pasar a este estado desde cualquier otro si ocurre un error.

De esta forma la definición se utiliza para la creación de la VM. ¿Cómo aseguramos la coherencia de los datos? Pues no dejando modificaciones en la definición de una VM que ya está creada. Para ser más precisos, habrá parámetros que se puedan modificar y otros no. Esto se analiza más adelante.

De esta forma, siempre habrá una correspondencia entre la definición, que está guardada en el descriptor, y la VM. Pero ¿qué pasa si alguien bypasea el orquestador y modifica directamente una VM en el hipervisor? La respuesta está en esta pregunta: ¿qué pasa con un fichero si alguien accede a los bloques de disco directamente? Pues puede crear incoherencia de datos, por eso, no se debe hacer. Aún así, en nuestro sistema, esta incoherencia es temporal, ya que, si el descriptor pasa a modo Provisioned, es decir, destruye la VM, la volverá a crear con la definición. Lo mismo ocurre al pasar de Stopped a Running, se podría habilitar la opción de que reescriba en el hipervisor, la configuración de los parámetros de la definición; así vuelve a haber coherencia. Esto son opciones a estudiar.

Características del orquestador

- Mantendrá una lista de todas las VM ,así como de sus definiciones. Todo unificado en el objeto “descriptor”, aunque conceptualmente, para un usuario es una VM “mejorada”
- Para poder manejar cientos de VM, se organizarán en forma de árbol. Esto permitirá realizar cualquier tipo de organización. No sólo la de asignaturas/alumnos/máquina, sino también cualquier otra necesidad: empresa/departamento/experimento/maquina, o lo que la se necesite. Esto da más universalidad. Los descriptors tendrán un nombre largo de la forma “/folder/subfolder/.../VMname”. Esta organización permitirá filtrar, mover copiar, etc.y más ventajas que veremos.
- Permitirá tener VM en diferentes hipervisores. Cada descriptor sabe dónde está la VM, en qué hipervisor. Es más, sabe el tipo de hipervisor, lo que permite tener diferentes hipervisores de diferentes tecnologías, unos pueden ser VMware, otros Proxmox, otros Oracle... El orquestador traducirá los métodos a llamadas a cada hipervisor usando un driver determinado para cada tipo.
- Como el descriptor puede estar en modo Provisioned, se puede hacer un despliegue “preliminar” de una infraestructura sin necesidad de usar los hipervisores reales. Esto también permitirá copiar y pegar, duplicar, etc. las máquinas Provisioned. Esto permite tener “carpetas” con copias de definiciones. También, tener máquinas “dummy” o “placeholders” por si en algún caso es útil.
- Al estar organizadas en árbol, se podrán tener definiciones comunes: ver más adelante: “definiciones heredadas en cascada”
- Tendrá control de usuarios. Existirán 3 roles:
 - Superusuario
 - Manager. Podrá crear carpetas, y descriptors
 - Usuario simple: Podrá arrancar, parar, crear y destruir VMs.
- Se podrán incluir parámetros personalizados, para ampliar la funcionalidad futura, o uso por parte de aplicaciones.
- Se podrá guardar las direcciones IP de las NIC de las MV

Parámetros de una VM.

Como hemos dicho, la definición de una VM es una lista de parámetros. Seguramente más que una simple lista, se organicen en grupos. Un diccionario de varios niveles, que se puede representar en json o en yaml fácilmente.

Hay que distinguir entre 2 tipos de parámetros, de creación y de ejecución

- Parámetros de creación.
 - Son los que se usan en la creación de una VM, como por ejemplo:
 - En qué hipervisor se crea
 - Nombre de VM
 - Imagen origen del disco
 - Ubicación de disco duro virtual
 - etc.
- Parámetros de ejecución
 - Estos parámetros también se usan en la creación de la VM, pero pueden ser modificados con la VM parada, como por ejemplo:
 - Cantidad de memoria
 - NICs y sus direcciones MAC
 - CDs virtuales conectados
 - En general, lo que los hypervisores dejan modificar con la máquina parada.

De esta forma tenemos claro que el descriptor, dejará modificar ciertos parámetros en función de su estado. Los de creación sólo se podrán modificar en estado Provisioned, y los de ejecución en estado Provisioned y Stopped.

Definiciones heredadas en cascada

Al haber permitido guardar los descriptores con una jerarquía de árbol, podemos aprovechar esto para crear un sistema de definiciones comunes.

Supongamos que tenemos una serie de VM que comparten características. Esto es muy común. Por ejemplo: compartirán hipervisor, compartirán datos de red, compartirán prototipos de máquina, etc. Para no tener que repetir todo eso en los parámetros de definición de cada descriptor, sería ideal tener una definición común. Pues esto es fácil de hacer con el árbol de descriptores. ¿Cómo?

- Cada carpeta tiene una “definición común”, una lista de parámetros comunes a esa carpeta.

- Cada descriptor tiene sus parámetros “locales”. Pues bien,
- La configuración computada, la que se usa para configurar la VM, es una fusión de los parámetros heredados de la carpeta, más los parámetros locales. Los locales sobreesciben los posibles comunes que hayan definidos.
- Los parámetros de la definición de la carpeta, también heredan previamente los de la carpeta padre.
- Así, la definición computada de una VM es una definición heredada en cascada desde la carpeta raíz. Cada carpeta va sobreescibiendo parámetros. Al final queda una definición computada completa.
- Esta definición computada se evalúa a demanda, cuando es necesaria usarla. Así, al modificar algún parámetro en un nivel anterior, se propaga su definición, hasta los descriptores.

Este sistema permite tener parámetros comunes por niveles. Por ejemplo, en el nivel raíz, se pueden definir hipervisores, cosas de res, prototipos, etc. Esto permite no tener que definirlos más, ya que se heredan. A cierto nivel se pueden definir otros prototipos, y solo se heredan en esa rama.

Al haber creado el sistema en árbol, si las propiedades de control de acceso se integran como otros parámetros, también se aprovechan de herencia en cascada. Así, se pueden definir políticas de acceso por carpetas, y niveles.

Creación de descriptores

- Creación directa. json o yaml. Para crear un descriptor, se le pasa una lista de parámetros. Como texto o fichero con formato json o yaml.
- Programas más sofisticados, como el que crea una infraestructura a partir de otro yaml más complejo. Estos programas, usando ficheros fuente, irán creando descriptores, carpetas, definiciones comunes.

¿qué ocurre con la coherencia aquí?

Han aparecido más fuentes de datos: los ficheros fuente que se usan para crear los descriptores. La idea es que estos ficheros se lean una vez y no tiene por qué estar sincronizados. Una vez creada una infraestructura, se podrá modificar. Otra cosa es que una aplicación quiera mantener esta coherencia. Le corresponde a esa aplicación, encargarse de ello. La idea es la misma, si algo se hace a más alto nivel, usar esa aplicación para todo, y no tocar a más bajo nivel.

Permisos

Habr  unas propiedades para definir permisos por cada recurso, o conjunto de recursos, aprovechando la estructura de  rbol.

Ver “detalles de implementaci n”

Direcciones IP

Diferentes modos, en principio, en la configuraci n de una VM no est  la IP. Es en la creaci n del descriptor donde se define la IP y el modo en que se usara. Habr  varios modos:

- DHCP normal
- DHCP, con IP codificada en la MAC. Requiere que el servidor DHCP tenga asignaciones est ticas definidas.
- IP fijada con Otros sistemas, como Cloud-init, o escritura directa de configuraci n en disco virtual.
- etc

Por tanto, la comunicaci n con el IPAM, recae principalmente en las aplicaciones que usan el m dulo orquestador.

Resumen

El orquestador mantiene descriptores que VM. Este interface de m s alto nivel, a ade al interface del hipervisor, m s informaci n; como estado Provisioned, informaci n de permisos, independencia del tipo de hipervisor. Y permite estructuras a medida, al ser un  rbol. Esto tambi n permite compartir y heredar propiedades.

M dulo de despliegue de asignaturas.

Es una aplicaci n concreta que, bas ndose en una definici n de infraestructura y listados de alumnos, despliega todas las VM sobre el orquestador. Este lee los ficheros de origen, crea una carpeta por la asignatura. Dentro, una carpeta por cada alumno. En cada carpeta de alumno, sus VMs.

Crea las definiciones comunes.

Asigna IPs con ayuda del IPAM.

Detalles de implementación

Posibles soluciones de implementación.

- Descriptores: queremos una estructura de datos permanente, en disco. Una base de datos o algo similar. Dado que queremos un árbol, una primera solución es utilizar directamente un directorio local y guardar los descriptores como ficheros, y las carpetas como carpetas. Es fácil y práctico. Es una solución de diseño interno, podría cambiar.
- Definiciones comunes. Ya que tenemos un sistema que guarda descriptores, la definición común de una carpeta se puede guardar como un descriptor especial en cada carpeta: `.../common`. Lógicamente este nombre queda reservado.
 - `/common`
 - `/folder1/common`
- Control de usuarios
 - Como primera solución, mantener un fichero con las listas de usuarios y grupos.
 - En cada descriptor (o definición común, que permite heredar), parámetros de permisos:
 - Lista de usuarios y grupos con rol de manager para ese recurso, y sus hijos que lo hereden.
 - Lista de usuarios con rol de `usuario_simple`
 - Permisos :
 - flags que indican qué operaciones están permitidas hacer a un manager, en ese recurso e hijos. Por ejemplo: puede ver, puede crear subcarpetas, puede definir nuevos prototipos, etc.
 - idem para `usuarios_simples`.
 - El acceso web / REST, aprovechará las tecnologías de autenticación existentes en http, para la identificación de usuarios: uso de tokens. Inicialmente BasicAuth. Futuro: OAuth2 Password Flow with JWTs
- Tecnologías:
 - REST API: FastAPI. httpx
 - Web UI: NiceGUI

- Pydantic y box para datos.. json, yaml...
- Rich para CLI
- Celery o RQ para tareas asíncronas (futuro)
- `ruamel.yaml`
- `Authlib` o similares

Fases de desarrollo

Desarrollo incremental. Cada etapa TTD.

- Creación de un hipervisor Mock, para poder desarrollar desacoplado de un hipervisor real.
- Módulo orquestador. Driver para hiper hipervisor mock. Inicialmente como librería. Creación de clase Descriptor. Funciones básicas. Creación desde json. Consultas básicas
- Ampliar funciones de orquestador. Guardado en árbol. Sin herencia. Aumentar métodos.
- Implementar herencia de propiedades en cascada
- como servicio. API REST, acceso remoto. Cliente CLI remoto.
- Implementar control de usuarios-permisos
- Adaptación de driver para MVware
- Módulo de despliegue de asignaturas. Adaptar el existente.
- Mejora de coordinación con IPAM
- Web gui.

PARa anteproyecto:

focalizar en docencia.